

Guida agli Script

OWASP Security Pipeline

Come usare gli script di routine per il test di sicurezza

Server: ubuntu@ta-server-openvas (51.75.196.150)

Directory script: ~/owasp-scan/

Team Tecnico | Versione 1.0 | Maggio 2026

1. Panoramica degli Script

Gli script di routine automatizzano le operazioni più frequenti del testing di sicurezza OWASP sulla VPS. Sono progettati per essere usati in sequenza — ogni script copre una fase specifica del workflow.

Script	Fase	Scopo	Durata tipica
01-start-env.sh	Preparazione	Avvia DVWA, ZAP daemon, Falco	~45 secondi
02-check-status.sh	Verifica	Controlla che tutti i servizi siano attivi	~5 secondi
03-run-scan.sh	Esecuzione	Lancia la pipeline owasp-scan.sh	Variabile
04-view-report.sh	Analisi	Avvia server HTTP per i report	Istantaneo
05-cleanup.sh	Chiusura	Ferma tutti i servizi	~10 secondi
06-analyze-vpn.sh	AI Analysis	Invia i report a Ollama via VPN per analisi	2-5 minuti

INFO Gli script si trovano in ~/owasp-scan/. Installarli con: `chmod +x ~/owasp-scan/*.sh`

2. Sequenza Operativa Consigliata

Seguire sempre questa sequenza per una sessione di test completa e affidabile.

```
# STEP 1 — Avvia l'ambiente
~/owasp-scan/01-start-env.sh

# STEP 2 — Verifica che tutto sia pronto
~/owasp-scan/02-check-status.sh

# STEP 3 — Lancia il test (scegli la modalita)
~/owasp-scan/03-run-scan.sh --dast-only      # solo Nikto + ZAP (15 secondi)
~/owasp-scan/03-run-scan.sh --dast-api      # + ASTF
~/owasp-scan/03-run-scan.sh                 # pipeline completa

# STEP 4 — Visualizza il report
~/owasp-scan/04-view-report.sh --latest

# STEP 5 — Analisi AI (opzionale, richiede VPN attiva)
~/owasp-scan/06-analyze-vpn.sh --latest

# STEP 6 — Chiudi l'ambiente
~/owasp-scan/05-cleanup.sh
```

3. Script di Dettaglio

01-start-env.sh Avvio Ambiente — Avvia DVWA, ZAP daemon e Falco

Prepara l'ambiente di test avviando tutti i servizi necessari in sequenza. Va eseguito all'inizio di ogni sessione di test. Controlla automaticamente se i servizi sono già attivi prima di riavviarli.

Cosa fa

- Monta il BPF filesystem (necessario per Falco su KVM — eseguito ad ogni avvio)
- Carica il kernel module Falco e avvia il processo di monitoring
- Avvia il container Docker DVWA sulla porta 8081
- Avvia ZAP in modalita daemon sulla porta 8080 (non webswing)
- Verifica che tutti i servizi rispondano correttamente prima di terminare

Sintassi

```
./01-start-env.sh          # avvia tutto
./01-start-env.sh --no-falco # salta Falco
./01-start-env.sh --no-zap   # salta ZAP
./01-start-env.sh --no-dvwa  # salta DVWA
```

Output atteso

```
[20:47:44] OK: BPF filesystem gia montato
[20:47:52] OK: Falco avviato (PID: 2161232)
[20:47:57] OK: DVWA avviato sulla porta 8081 (HTTP 302)
[20:48:27] OK: ZAP daemon avviato (versione: 2.17.0, atteso 30s)
```

ATTENZIONE ZAP impiega 20-30 secondi ad avviarsi. Lo script attende automaticamente fino a 60 secondi prima di segnalare un errore.

Servizio	Porta	Verifica	Stato atteso
DVWA	8081	curl -s -o /dev/null -w "%{http_code}" http://localhost:8081	302
ZAP	8080	curl -s http://localhost:8080/JSON/core/view/version/?apikey=...	{"version":"2.17.0"}
Falco	—	ps aux grep falco grep -v grep	processo attivo

02-check-status.sh Verifica Stato — Controlla tutti i servizi e strumenti

Esegue una verifica completa di tutti i componenti dello stack prima di lanciare un test. Produce un riepilogo con PASS, WARN e FAIL per ogni controllo. Uscita con exit code 1 se ci sono FAIL critici.

Cosa controlla

Controllo	Cosa verifica	Critico
Docker	Versione e daemon attivo	Si
DVWA	Container attivo e risposta HTTP 302	Si
ZAP	Container attivo e API REST in modalita daemon	Si
Falco	Processo attivo, BPF montato, kernel module	Si
ASTF	JAR presente e Java 21 installato	Si
WuppieFuzz	Binario nel PATH	No
Semgrep	Immagine Docker disponibile	No
GitLeaks	Immagine Docker disponibile	No
Dependency-Check	Immagine Docker disponibile	No
Nikto	Comando disponibile	No
owasp-scan.sh	Script presente, eseguibile, sintassi corretta	Si
config.env	File presente con TARGET_URL configurato	Si

Sintassi

```
./02-check-status.sh           # verifica completa con output dettagliato
./02-check-status.sh --quiet   # mostra solo FAIL e WARN
```

Interpretare il risultato

```
=====
RIEPILOGO: 21 pass | 0 warn | 0 fail
=====
Ambiente pronto per il test.
```

- 21 pass, 0 fail: ambiente pronto, si puo procedere con il test
- FAIL presenti: eseguire 01-start-env.sh per avviare i servizi mancanti
- Solo WARN: ambiente funzionante ma con limitazioni — valutare prima di procedere

03-run-scan.sh Lancio Pipeline — Esegue owasp-scan.sh con la modalita scelta

Lancia la pipeline owasp-scan.sh con controlli pre-avvio automatici. Verifica che DVWA e ZAP siano raggiungibili prima di partire. Monitora l'output in tempo reale e segnala il percorso del report al termine.

Modalita disponibili

Flag	Strumenti eseguiti	Durata tipica	Quando usarla
(nessuno) default	Nikto + ZAP	~15 secondi	Verifica rapida quotidiana
--dast-only	Nikto + ZAP	~15 secondi	Solo test web, senza API
--api-only	ASTF + WuppieFuzz	5-15 minuti	Solo test API REST
--dast-api	Nikto + ZAP + ASTF	10-20 minuti	Test web + API completo
--full	Tutti gli strumenti	30-60 minuti	Pentest trimestrale
--tool NOME	Uno strumento specifico	Variabile	Debug o test singolo

Sintassi

```
./03-run-scan.sh           # DAST rapido (default)
./03-run-scan.sh --dast-only # solo Nikto + ZAP
./03-run-scan.sh --dast-api  # web + API
./03-run-scan.sh --full      # tutto
./03-run-scan.sh --tool nikto # solo Nikto
./03-run-scan.sh --tool zap   # solo ZAP
./03-run-scan.sh --tool astf  # solo ASTF
```

Strumenti disponibili per --tool

```
nikto | zap | astf | wuppiefuzz | semgrep | gitleaks | depcheck |
openssl
```

Output atteso

```
[15:00:10] Avvio nikto...
nikto completato in 8s
[15:00:19] Avvio ZAP spider su http://172.17.0.1:8081...
[15:00:24] Avvio ZAP active scan...
ZAP completato - 276 alerts trovati
[15:00:37] Pipeline completata
[15:00:37] Report disponibile in: ~/owasp-scan/reports/20260515_150010/
[15:00:37] Per visualizzarlo: ./04-view-report.sh
```

ATTENZIONE Prima di lanciare verificare sempre che ZAP sia in modalita DAEMON. Se ZAP e in modalita webswing lo script fallisce silenziosamente. Usare 02-check-status.sh per verificare.

04-view-report.sh Visualizzazione Report — Avvia server HTTP per i report nel browser

Avvia un server HTTP Python nella directory del report selezionato. Elenca tutti i report disponibili con data, numero di file e alert ZAP trovati. Permette di scegliere quale report visualizzare.

Sintassi

```
./04-view-report.sh # elenca e chiede quale aprire
./04-view-report.sh --latest # apre direttamente l'ultimo report
./04-view-report.sh --list # elenca i report senza avviare il server
./04-view-report.sh --port 8888 # usa porta diversa da 9999
```

Come usarlo

```
# 1. Lancia lo script
./04-view-report.sh

# 2. Seleziona il report dalla lista
=== Report disponibili ===
[1] 15/05/2026 15:00:10 | ZAP alerts: 276
[2] 13/05/2026 20:50:12 | ZAP alerts: 203
Seleziona report (invio per il piu recente): 1

# 3. Apri nel browser
http://51.75.196.150:9999/index.html # report unificato
http://51.75.196.150:9999/ai-analysis.html # analisi AI
http://51.75.196.150:9999/nikto-report.html # report Nikto
http://51.75.196.150:9999/zap-alerts.json # alert ZAP raw

# 4. Premi Ctrl+C per fermare il server
```

File presenti in ogni report

File	Contenuto	Generato da
index.html	Report unificato HTML con tutti i findings	owasp-scan.sh
ai-analysis.html	Analisi AI — sommario IT + dettaglio EN	06-analyze-vpn.sh
nikto-report.html	Report HTML Nikto	Nikto
nikto.log	Log grezzo Nikto	Nikto
zap-alerts.json	Alert ZAP in formato JSON	ZAP
zap.log	Log comunicazione con API ZAP	owasp-scan.sh
astf-report.html	Report HTML ASTF — OWASP API Top 10	ASTF
semgrep-report.sarif	Findings SAST in formato SARIF	Semgrep
gitleaks-report.sarif	Secrets trovati in formato SARIF	GitLeaks
dependency-check/	Report SCA con CVE nelle dipendenze	Dependency-Check

05-cleanup.sh Pulizia Ambiente — Ferma tutti i servizi a fine sessione

Ferma in modo ordinato tutti i servizi e processi avviati durante la sessione. Rimuove i container Docker, killa i processi residui e opzionalmente pulisce i report vecchi.

Cosa ferma

- Container Docker ZAP
- Container Docker DVWA
- Processo Falco
- Processi ASTF (Java) residui
- Processi WuppieFuzz residui
- Processo owasp-scan.sh se in esecuzione
- Server HTTP report (python3 -m http.server)
- Immagini Docker dangling

Sintassi

```
./05-cleanup.sh          # ferma servizi, mantieni i report
./05-cleanup.sh --reports # ferma tutto + cancella report > 7 giorni
./05-cleanup.sh --all    # ferma tutto + cancella TUTTI i report
./05-cleanup.sh --no-falco # ferma tutto tranne Falco
```

Output atteso

```
[OK] ZAP fermato e rimosso
[OK] DVWA fermato
[OK] Falco fermato
[SKIP] Nessun processo ASTF attivo
[SKIP] Nessun server HTTP report attivo

=== Stato finale ===
Container Docker attivi:
    nessuno
Processi falco attivi:
    nessuno
```

TIP Usare --reports periodicamente per liberare spazio disco — ogni scan genera 10-50 MB di report. I report più recenti vengono sempre mantenuti.

06-analyze-vpn.sh Analisi AI — Invia i findings a Ollama sulla PGX via WireGuard

Invia i findings del report a Ollama (PGX ThinkStation via WireGuard VPN) per un'analisi AI approfondita. Il modello produce un report HTML con sommario in italiano e dettaglio tecnico in inglese, prioritizzazione delle vulnerabilità e piano di remediation.

Prerequisiti

- WireGuard VPN attiva verso la PGX (sudo wg show deve mostrare handshake recente)
- Ollama in esecuzione sulla PGX con OLLAMA_HOST=0.0.0.0

- Modello scaricato sulla PGX (es. qwen2.5:32b o qwen2.5:72b)
- Almeno un report generato da owasp-scan.sh in ~/owasp-scan/reports/

Sintassi

```
./06-analyze-vpn.sh # lista report e chiede quale
analizzare
./06-analyze-vpn.sh --latest # analizza l'ultimo report
./06-analyze-vpn.sh --model qwen2.5:72b # usa modello specifico
./06-analyze-vpn.sh --model qwen2.5:32b --latest
./06-analyze-vpn.sh --report ~/owasp-scan/reports/20260515_150010/
```

Struttura del report AI generato

Sezione	Lingua	Contenuto
Sommario Esecutivo	Italiano	8-10 righe — rischio generale, finding principali, azioni urgenti
Risk Level	Italiano	CRITICO / ALTO / MEDIO / BASSO
Top 10 Vulnerabilities	Inglese	Prioritizzate per exploitability con CWE e flag falsi positivi
Probable False Positives	Inglese	Alert che probabilmente non sono vulnerabilita reali
Remediation Plan	Inglese	Azioni ordinate per priorita con effort e owner
Technical Details	Inglese	Per ogni finding critico: scenario di attacco e fix con codice
What Was Not Found	Inglese	Classi di vulnerabilita non coperte dallo scan

Configurazione Ollama e modelli consigliati

Modello	Dimensione	Qualita	Velocita	Note
qwen2.5:72b	72B	Eccellente	Lenta (5-10 min)	Prima scelta — nessun thinking mode
qwen2.5:32b	32B	Ottima	Media (2-5 min)	Buon compromesso qualita/velocita
llama3.3:70b	70B	Ottima	Media (4-8 min)	Alternativa valida a Qwen
qwen3.6:35b	35B	Ottima	Lenta (10+ min)	Thinking mode — molta latenza

ATTENZIONE I modelli con thinking mode (qwen3.x, gpt-oss, deepseek-r1) producono migliaia di chunk interni prima di rispondere — tempi molto piu lunghi. Preferire qwen2.5 senza thinking per questo caso d'uso.

Connessione VPN

```
# Verifica VPN attiva
sudo wg show
ping -c 2 10.0.0.1

# Se VPN non attiva
```

```
sudo wg-quick up wg0

# Verifica Ollama sulla PGX
curl -s http://10.0.0.1:11434/api/tags | python3 -c "import sys,json;
[print(m['name']) for m in json.load(sys.stdin).get('models',[])]"
```

Flusso di esecuzione

```
1. Verifica VPN WireGuard attiva
2. Verifica Ollama raggiungibile su 10.0.0.1:11434
3. Verifica modello disponibile sulla PGX
4. Seleziona report da analizzare
5. Estrae findings da ZAP, Nikto, ASTF (max 50 alert)
6. Sanitizza caratteri speciali dal JSON
7. Costruisce prompt strutturato e lo salva su file
8. Invia a Ollama via curl con streaming
9. Raccoglie i token (response + thinking se presente)
10. Genera report HTML con analisi formattata
11. Salva ai-analysis.html nella directory del report
```

4. Lo Script Principale — owasp-scan.sh

owasp-scan.sh è lo script orchestratore che viene chiamato da 03-run-scan.sh. Può essere usato anche direttamente per maggiore controllo sulle opzioni.

Configurazione — config.env

L'unico file da modificare per cambiare target o credenziali. Non toccare mai owasp-scan.sh direttamente.

Variabile	Scopo	Esempio
PROJECT_NAME	Nome del progetto nel report	"dvwa-test"
TARGET_URL	URL dell'app web da testare	"http://172.17.0.1:8081"
TARGET_API_URL	URL base delle API REST	"http://172.17.0.1:8081"
SOURCE_DIR	Directory codice sorgente (SAST/SCA)	"/home/ubuntu/dvwa-source"
ZAP_PORT	Porta API REST di ZAP	"8080"
ZAP_API_KEY	API key ZAP	"dallaradallara20262026"
NVD_API_KEY	API key NVD per Dependency-Check	"ef5e1703-..."
ASTF_TIMEOUT	Timeout ASTF in minuti	"10"
ASTF_THREADS	Thread paralleli ASTF (max 8 su 4 core)	"8"
OPENAPI_SPEC	Path spec OpenAPI per WuppieFuzz	"" (vuoto = skip)
OPENVAS_HOST	IP macchina OpenVAS	"" (vuoto = skip)
AUTO_SERVE	Avvia server HTTP automatico	"false"

Flag della pipeline

Flag	Fase saltata	Quando usarlo
--skip-secrets	GitLeaks	Nessun repository git disponibile
--skip-sast	Semgrep	Nessun codice sorgente disponibile
--skip-sca	Dependency-Check	Nessun codice sorgente disponibile
--skip-dast	Nikto + ZAP	Solo test API, no web app
--skip-api	ASTF + WuppieFuzz	Solo test web, no API
--skip-opensvas	OpenVAS	No scan di rete necessario
--only TOOL	Tutti tranne uno	Test o debug di un singolo strumento

5. Casi d'Uso Comuni

5.1 Test rapido quotidiano su DVWA

Verifica veloce per monitorare lo stato della sicurezza. Circa 1 minuto totale.

```
./02-check-status.sh --quiet      # verifica che tutto sia ok
./03-run-scan.sh --dast-only      # Nikto + ZAP (~15 secondi)
./04-view-report.sh --latest      # apri il report
```

5.2 Test completo con analisi AI

Pipeline completa con report finale analizzato dal modello. Circa 15-30 minuti totali.

```
./01-start-env.sh
./02-check-status.sh
./owasp-scan.sh --skip-api --skip-openssl      # GitLeaks + Semgrep + Dep-Check +
Nikto + ZAP
./06-analyze-vpn.sh --latest                  # analisi AI via VPN
./04-view-report.sh --latest
./05-cleanup.sh
```

5.3 Test su nuovo target

Prima di testare un target diverso da DVWA aggiornare config.env.

```
# 1. Modifica config.env
nano ~/owasp-scan/config.env
# Cambia: TARGET_URL, TARGET_API_URL, TARGET_IP
# Cambia: SOURCE_DIR se hai il codice sorgente
# Cambia: API_TOKEN se l'API richiede autenticazione

# 2. Lancia la pipeline
./03-run-scan.sh --dast-api

# 3. Analisi AI
./06-analyze-vpn.sh --latest
```

5.4 Solo analisi AI di un report esistente

Se hai già un report e vuoi solo l'analisi del modello senza rieseguire lo scan.

```
./06-analyze-vpn.sh --list          # vedi i report disponibili
./06-analyze-vpn.sh --report ~/owasp-scan/reports/20260515_150010/
./04-view-report.sh --latest        # visualizza il report con l'analisi
```

5.5 Pentest trimestrale completo

Sequenza completa per il pentest periodico. Richiedere SOURCE_DIR configurato con il codice sorgente.

```
# 1. Prepara il sorgente
git clone https://github.com/REPO/app.git ~/source

# 2. Aggiorna config.env
nano ~/owasp-scan/config.env
# TARGET_URL, SOURCE_DIR, API_TOKEN, OPENVAS_HOST

# 3. Pipeline completa
```

```
./01-start-env.sh
./02-check-status.sh
./owasp-scan.sh # tutto

# 4. Analisi AI
./06-analyze-vpn.sh --model qwen2.5:72b --latest

# 5. Report
./04-view-report.sh --latest
./05-cleanup.sh --reports
```

6. Troubleshooting

Problema	Causa	Soluzione
ZAP non risponde	Container fermo o in modalita webswing	<code>docker stop zap && docker rm zap</code> <code>./01-start-env.sh</code>
02-check-status: ZAP FAIL	ZAP in webswing invece di daemon	Riavviare con <code>zap.sh -daemon</code> — vedi 01-start-env.sh
03-run-scan si interrompe a ZAP	ZAP fermo durante lo scan	Verificare <code>docker ps grep zap</code> durante lo scan
Falco: Trying to open engine	BPF filesystem non montato	<code>sudo mount -t bpf bpf /sys/fs/bpf</code>
06-analyze-vpn: VPN non trovata	WireGuard non attivo	<code>sudo wg-quick up wg0</code>
06-analyze-vpn: risposta vuota	Modello con thinking mode	Usare <code>qwen2.5:32b</code> o <code>qwen2.5:72b</code> invece di <code>qwen3.x</code>
06-analyze-vpn: timeout	Modello troppo lento o VPN instabile	Aumentare <code>TIMEOUT_SEC</code> nello script o usare modello piu piccolo
Dep-Check: timeout NVD	Rate limit API NVD	Aggiungere <code>--nvdApiDelay 6000</code> in <code>owasp-scan.sh</code>
ASTF: report non generato	Processo killato prima della fine	Usare <code>nohup</code> o aumentare <code>ASTF_TIMEOUT</code> in <code>config.env</code>
WuppieFuzz: command not found	PATH cargo non caricato	<code>source ~/.cargo/env</code>

7. Note Importanti

Sicurezza degli script

Gli script eseguono tool che ATTACCANO ATTIVAMENTE il target. Usarli solo su sistemi di cui si e proprietari o su cui si ha autorizzazione scritta. Su DVWA si puo procedere liberamente — e progettato per essere attaccato.

ZAP daemon vs webswing

06-analyze-vpn.sh e owasp-scan.sh richiedono ZAP in modalita daemon. La modalita webswing (interfaccia grafica) non espone l'API REST sulla stessa porta. 01-start-env.sh avvia automaticamente ZAP in daemon.

BPF su KVM

La VPS e una VM KVM. Il BPF filesystem deve essere montato ad ogni riavvio per Falco. E gia aggiunto a `/etc/fstab` — dovrebbe montarsi automaticamente. In caso contrario: `sudo mount -t bpf /sys/fs/bpf`

Modelli AI consigliati

Per l'analisi AI preferire qwen2.5:32b o qwen2.5:72b — non hanno thinking mode e rispondono in 2-5 minuti. I modelli qwen3.x e gpt-oss hanno thinking mode che produce migliaia di token interni prima di rispondere.

Archiviazione report

Ogni esecuzione di owasp-scan.sh crea una nuova directory con timestamp in ~/owasp-scan/reports/. I report crescono nel tempo — usare ./05-cleanup.sh --reports periodicamente per liberare spazio.